eBPF

Another piece of technology that many (open source) continuous profiling tools use is the Linux kernel feature eBPF. Nowadays, it is used as a term in itself, but originally, the term derives from *Berkeley Packet Filter* (BPF).

Linux kernel v3.18 (released on December 7, 2014) introduced support for eBPF. This kernel version requirement and the fact that you need good support for debug symbols to meaningfully use it means eBPF is, at least at the time of writing in early 2023, still restricted to certain environments, including dev/test (where you have full control over the Linux version used, in contrast to production environments, where using debug symbols slows down execution).

> **NOTE** You might wonder why I'm saying that eBPF is restricted to certain environments when looking at the kernel version in which it was introduced. At time of writing, it has been almost 10 years since that happened; however, cloud providers and large enterprises take a little longer to move their default Linux versions to "newer" kernels, due to distributions or otherwise. Ask your platform team if you can use eBPF in prod, but don't be surprised if the answer is, "We're working on it."

Now, why do we need eBPF, and how does it work? It enables you to safely and efficiently extend the Linux kernel by adding new functionality as you see fit. It is exposed to the Linux user space via the `bpf(2)` syscall (see also the man page [https://man7.org/linux/man-pages/man2/bpf.2.html] for further details) and implemented in the kernel as a virtual machine (VM) that defines a custom 64-bit RISC instruction set. Figure 9.3 shows a high-level overview of how eBPF works, from the book *BPF Performance Tools* by Brendan Gregg (2019, Addison-Wesley, http://mng.bz/1q1j). If you want to learn more about eBPF, I highly recommend reading *What Is eBPF?* by Liz Rice (2022, O'Reilly, http://mng.bz/Pzln).

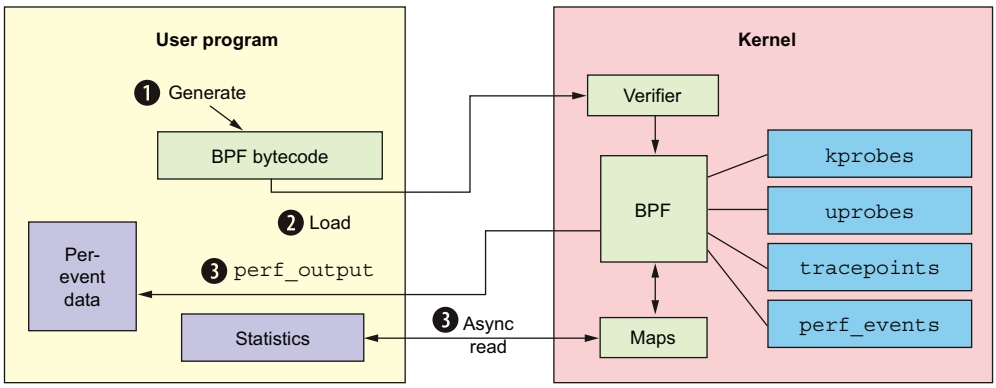


Figure 9.3 The eBPF call flow in the Linux kernel at a conceptual level (Source: Brendan Gregg. Licensed under CC BY 4.0)